

Conv-ChArT — deployment cost

One architecture, two deployment points. `width_mult` scales every convolution channel **and** the attention dimension together, so the conv:attention split is preserved exactly as the model shrinks — these are a self-similar rescaling of one design, not two separately tuned networks. Both are smaller than the baseline they are measured against.

model	detector	+ refiner	vs Deep ChArUco	corner ID	loc. coarse (px)	loc. refined (px)	GFLOPs / frame	Orin fp16	fps	headroom
Conv-ChArT	882,402	979,458	0.44x	98.95%	0.455	0.242	52.0	5.0 ms	198	13x
Conv-ChArT-FAST	222,138	319,194	0.14x	90.93%	0.475	0.238	19.8	1.9 ms	522	34x

Reference: Deep ChArUco totals **2,241,235** parameters (ChArUcoNet 1,242,002 + RefineNet 999,233, counted from their source) at a converged **85.44%** corner-ID accuracy. The **vs Deep ChArUco** column compares *totals* against *total*, which is the only like-for-like reading — our detector alone against their detector-plus-refiner would flatter us.

Reading the table

- **The refiner is a fixed 97,056 parameters, shared unchanged by both models.** It is 9.9% of Conv-ChArT but 30.4% of Conv-ChArT-FAST, so at the small end it is the dominant remaining cost — the cheapest further saving there is a narrower refiner, not a narrower detector.
- **corner ID is the final 10,000-sample validation**, not the 2,000-sample one that runs every 2,500 steps: the small val reads ~0.1–0.2 pp optimistic (Conv-ChArT 99.04% on 2k vs 98.95% on 10k). It is M-04, directly comparable to Deep ChArUco’s 85.44%.
- **The two localisation columns are MEAN error, pooled over every factor and step of the robustness sweep** — across the whole tested envelope, not a benign nominal condition. They come from the sweep rather than the training validation because `run_validation` scores the COARSE peaks only and never runs the refiner, so no with-refiner number exists there. On the same pooled basis Deep ChArUco is **1.591 px** and classical **0.741 px**; both are unrefined, and Deep ChArUco’s own RefineNet is not run, so their column belongs against our COARSE one.
- **Localisation is conditioned on detection** (matched pairs within 4 px), so a detector that finds only its easiest corners looks precise. Read it beside recall, never alone.
- **headroom is against the 66 ms frame budget** (15 Hz pose = 30 fps of lit/dark differencing pairs). Both models clear it comfortably.
- **FAST is a COST argument, not a speed one.** Conv-ChArT already clears the latency target with room to spare, so what FAST buys is reach: at 19.8 GFLOPs it opens Orin NX- and Nano-class modules, plus lower energy per frame and more SoC left for planning and control.
- **The Orin column is a datasheet roofline, not a measurement.** GFLOPs/frame at 15% of the module’s fp16 peak; at an optimistic 30% the same graphs give Conv-ChArT 396, Conv-ChArT-FAST 1043 fps. No TensorRT engine has been built and nothing has run on Orin hardware — these bound the compute, not the latency.
- **Parameter count is the size axis, not the memory axis.** Measured at batch 1, weights are 1.0–3.7 MB across these two while activations are 61.5–122.4 MB, so memory is dominated by activations, which scale with input resolution rather than width.